

```
#17/08/26
import pandas as pd
employees = {
    "EmpID" : [101,102,103,104],
    "Name" : ["Asha",None,"Menna","Sai"],
    "Deptmnt" : ["Sales","IT","HR",None],
    "Salary" : [45000,60000,52000,89000]
}
i = [901,902,903,904]
df = pd.DataFrame(employees,index = i)
#1.create an dataframe
print(df)
```

	EmpID	Name	Deptmnt	Salary
901	101	Asha	Sales	45000
902	102	None	IT	60000
903	103	Menna	HR	52000
904	104	Sai	None	89000

Double-click (or enter) to edit

```
#2.first 2 rows and lastrows 2
print(df.head(2))
print(df.tail(2))
```

	EmpID	Name	Deptmnt	Salary
901	101	Asha	Sales	45000
902	102	None	IT	60000

	EmpID	Name	Deptmnt	Salary
903	103	Menna	HR	52000
904	104	Sai	None	89000

```
#3.display only sal and name col
print(df[["Name","Salary"]])
```

	Name	Salary
901	Asha	45000
902	None	60000
903	Menna	52000
904	Sai	89000

```
#4.display the emp empid = 103
print(df[df["EmpID"] == 103])
```

	EmpID	Name	Deptmnt	Salary
903	103	Menna	HR	52000

```
#5.diaplay all the emp whose sal > 50000
print(df[df["Salary"] > 50000])
```

	EmpID	Name	Deptmnt	Salary
902	102	None	IT	60000
903	103	Menna	HR	52000
904	104	Sai	None	89000

```
#6. display emp whose belong to IT deprtm
print(df[df["Deptmnt"] == "IT"])
```

	EmpID	Name	Deptmnt	Salary
902	102	None	IT	60000

```
#7.display emp whose name or dprt containg a missing value
print(df[df["Name"].isna() | df["Deptmnt"].isna()])
```

	EmpID	Name	Deptmnt	Salary
902	102	None	IT	60000
904	104	Sai	None	89000

```
#8.find the missing values in ech col
print(df.isna().sum())
```

```
EmpID      0
Name       1
Deptmnt    1
Salary     0
dtype: int64
```

```
#9.fill the missing name with "unkown" and deptrt with "not assigned"
print(df.fillna({"Name" : "unkown","Deptrmnt" : "Not assigned"}))
```

	EmpID	Name	Deptrmnt	Salary
901	101	Asha	Sales	45000
902	102	unkown	IT	60000
903	103	Menna	HR	52000
904	104	Sai	Not assignes	89000

```
#10.add a new col call bonus, where the bonus is 10% of salary
k = df['Bonus'] = df['Salary'] * 0.10
print(k)
print(df)
```

901	4500.0
902	6000.0
903	5200.0
904	8900.0

Name: Salary, dtype: float64

	EmpID	Name	Deptrmnt	Salary	Bonus
901	101	Asha	Sales	45000	4500.0
902	102	None	IT	60000	6000.0
903	103	Menna	HR	52000	5200.0
904	104	Sai	None	89000	8900.0

```
#11.update sai sal from 89000 to 95000
import pandas as pd
employees = {
    "EmpID" : [101,102,103,104],
    "Name" : ["Asha",None,"Menna","Sai"],
    "Deptrmnt" : ["Sales","IT","HR",None],
    "Salary" : [45000,60000,52000,89000]
}
i = [901,902,903,904]
df = pd.DataFrame(employees,index = i)

#11.update sai sal from 89000 to 95000
#where
n = df["Salary"].where(df["Name"]!= "Sai",95000)
print(n)

df.loc[df["Name"]=="Sai","Salary"] = 95000
print(df)

df.loc[df["Name"]=="Sai","Sai"] = 95000
print(df)
```

901	45000
902	60000
903	52000
904	95000

Name: Salary, dtype: int64

	EmpID	Name	Deptrmnt	Salary
901	101	Asha	Sales	45000
902	102	None	IT	60000
903	103	Menna	HR	52000
904	104	Sai	None	95000

	EmpID	Name	Deptrmnt	Salary	Sai
901	101	Asha	Sales	45000	NaN
902	102	None	IT	60000	NaN
903	103	Menna	HR	52000	NaN
904	104	Sai	None	95000	95000.0

```
#12.rename the col EmpID to EmployeeID
df.rename (columns= {
    "EmpId" : "Employee_ID",
    "Name" : "Name",
    "Deptrmnt" : "Deptrmnt",
    "Salary" : "Salary",
    "Bonus" : "Bonus"
},
inplace=True)
print(df)
```

	EmpID	Name	Deptrmnt	Salary	Bonus
901	101	Asha	Sales	45000	4500.0
902	102	None	IT	60000	6000.0
903	103	Menna	HR	52000	5200.0
904	104	Sai	None	89000	8900.0

```
#13.del the bonus
print(df.drop(columns=["Bonus"]))
```

	EmpID	Name	Dept	Salary
901	101	Asha	Sales	45000
902	102	None	IT	60000
903	103	Meena	HR	52000
904	104	Sai	None	89000

```
#14.
m = df["Salary"].min()
print(m)

print(df["Salary"].max())

print(df["Salary"].mean())

print(df["Salary"].sum())
```

```
45000
89000
61500.0
246000
```

```
#15.find total sal
print(df[""])
```

```
#16
```

```
#17
emp1={
    "EmpID": [101,102,103,104],
    "Name": ["Asha",None,"Meena","Sai"],
    "Department": ["Sales","IT","HR",None],
    "Salary": [45000,60000,52000,89000]
}
i=[901,902,903,904]
df1=pd.DataFrame(emp1,index=i)

emp2={
    "EmpID": [101,102,103,104],
    "City": ["Hyderabad","Chennai","Bangalore","Pune"],
    "Experience": [2,4,3,5]
}
i=[901,902,903,904]
df2=pd.DataFrame(emp2,index=i)

#inner
m=pd.merge(df1,df2,on='EmpID',how='inner')
print(m)

#outer
m=pd.merge(df1,df2,on='EmpID',how='outer')
print(m)

#right
m=pd.merge(df1,df2,on='EmpID',how='right')
print(m)

#left
m=pd.merge(df1,df2,on='EmpID',how='left')
print(m)
```

	EmpID	Name	Department	Salary	City	Experience
0	101	Asha	Sales	45000	Hyderabad	2
1	102	None	IT	60000	Chennai	4
2	103	Meena	HR	52000	Bangalore	3
3	104	Sai	None	89000	Pune	5
	EmpID	Name	Department	Salary	City	Experience
0	101	Asha	Sales	45000	Hyderabad	2
1	102	None	IT	60000	Chennai	4
2	103	Meena	HR	52000	Bangalore	3
3	104	Sai	None	89000	Pune	5
	EmpID	Name	Department	Salary	City	Experience
0	101	Asha	Sales	45000	Hyderabad	2
1	102	None	IT	60000	Chennai	4

2	103	Meena	HR	52000	Bangalore	3
3	104	Sai	None	89000	Pune	5

```
#19. create a new data frame containg emply 105 and 106 and use pd.concat() to add them def_emp
new_data = {
    "EmpID" : [105,106],
    "Name" : ["Ravi","priya"],
    "Department" : ["HR","Sales"],
    "Salary" : [90000,89000]
}
df_new = pd.DataFrame(new_data)
df_emp = pd.concat([df1,df_new],ignore_index = True)
print(df_emp)
```

	EmpID	Name	Department	Salary
0	101	Asha	Sales	45000
1	102	None	IT	60000
2	103	Meena	HR	52000
3	104	Sai	None	89000
4	105	Ravi	HR	90000
5	106	priya	Sales	89000

```
#fixing data type - price should be numeric - change one data type into another data type , - int - float, str - str
df["Salary"] = df["Salary"].astype(float)
print(df)

df["Name"] = df["Name"].astype(str)
print(df)
```

	EmpID	Name	Deptmmt	Salary	Bonus
901	101	Asha	Sales	45000.0	4500.0
902	102	None	IT	60000.0	6000.0
903	103	Menna	HR	52000.0	5200.0
904	104	Sai	None	89000.0	8900.0

	EmpID	Name	Deptmmt	Salary	Bonus
901	101	Asha	Sales	45000.0	4500.0
902	102	None	IT	60000.0	6000.0
903	103	Menna	HR	52000.0	5200.0
904	104	Sai	None	89000.0	8900.0

```
#creating an new col
df["total"] = df["price"] * df["quantity"]
```

```
#unique() and nunique() - a particular row must be same a - 10,20,30 , a-10,20,30
n = {
    "EmpID": [101,102,103,104,105],
    "City": ["Hyderabad","Chennai","Pune","Bangalore","Pune"],
    "Experience": [2,4,3,5,6]
}
df = pd.DataFrame(n)
print(df["City"].unique())
print(df["City"].nunique())
```

```
['Hyderabad' 'Chennai' 'Pune' 'Bangalore']
4
```

```
#where - condition
n = df["Salary"].where(df["Name"] != "Sai",95000)
print(n)

df.loc[df["Name"]=="Sai","Salary"] = 95000
print(df)

df.loc[df["Name"]=="Sai","Sai"] = 95000
print(df)
```

901	45000
902	60000
903	52000
904	95000

Name: Salary, dtype: int64

	EmpID	Name	Deptmmt	Salary	Sai
901	101	Asha	Sales	45000	NaN
902	102	None	IT	60000	NaN
903	103	Menna	HR	52000	NaN
904	104	Sai	None	95000	95000.0

	EmpID	Name	Deptmmt	Salary	Sai
901	101	Asha	Sales	45000	NaN
902	102	None	IT	60000	NaN
903	103	Menna	HR	52000	NaN
904	104	Sai	None	95000	95000.0

```
#19-8-26
#Finding the duplicate values --df.duplicated()
import pandas as pd
data = {
    "EmpID" : [101,102,103,104,105,106,107,108,109,110,110],
    "Name" : ["Ravi","PRIYA","Arun","sneha","Kiran","ANJALI","Vikram","meena","RAHUL","sneha","sneha"],
    "City" : ["Hyderabad","Hyderabad","CHENNAI","Chennai","Bangalore","BANGALORE","Mumbai","mumbai","Delhi","Hyderabad","Hyderabad"],
    "Salary" : [30000,40000,3500,4500,32000,50000,38000,42000,36000,45000,45000],
}
df = pd.DataFrame(data)
print(df)
```

	EmpID	Name	City	Salary
0	101	Ravi	Hyderabad	30000
1	102	PRIYA	Hyderabad	40000
2	103	Arun	CHENNAI	3500
3	104	sneha	Chennai	4500
4	105	Kiran	Bangalore	32000
5	106	ANJALI	BANGALORE	50000
6	107	Vikram	Mumbai	38000
7	108	meena	mumbai	42000
8	109	RAHUL	Delhi	36000
9	110	sneha	Hyderabad	45000
10	110	sneha	Hyderabad	45000

```
#find duplicate values -- to check whether dupl is there or not if it print yes then dupl exists
print(df.duplicated())
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9    False
10    True
dtype: bool
```

```
#to count duplicate values -- helps to count the duplicate values
count = df.duplicated().sum()
print(count)
```

```
1
```

```
#display the actual duplicate rows -- display the dupl rows
print(df[df.duplicated()])
```

	EmpID	Name	City	Salary
10	110	sneha	Hyderabad	45000

```
#find all the rows involved in a duplicate
df.duplicated()
df[df.duplicated(keep=False)]
#or
duplicates = df[df.duplicated(keep=False)]
print(duplicates)
```

	EmpID	Name	City	Salary
9	110	sneha	Hyderabad	45000
10	110	sneha	Hyderabad	45000

```
#get sum of duplicate rows
duplicate_rows = df[df.duplicated()]
total = duplicate_rows["Salary"].sum()
print(total)
```

```
45000
```

```
#original and duplicate sum -- 45000 + 45000 = 90000
duplicate_rows = df[df.duplicated(keep=False)]
total = duplicate_rows["Salary"].sum()
print(total)
```

```
90000
```

```
#find duplicates in a specific column
#primary key, foreign, unique, composite, super, candidate, surrogate
print(df["EmpID"].duplicated())
#or
print(df[df["EmpID"].duplicated()])
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9    False
10   True
Name: EmpID, dtype: bool
   EmpID  Name      City  Salary
10    110  sneha  Hyderabad  45000
```

```
#display rows having duplicate employee ids
df[df["EmpID"].duplicated()]
```

	EmpID	Name	City	Salary
10	110	sneha	Hyderabad	45000

```
#find
df[df["EmpID"].duplicated(keep=False)]
```

	EmpID	Name	City	Salary
	110	sneha	Hyderabad	45000

```
#find duplicate of a specific names
df[df["Name"].duplicated()]
```

	EmpID	Name	City	Salary
9	110	sneha	Hyderabad	45000
10	110	sneha	Hyderabad	45000

```
#drop duplicates
#drop all the duplicates rows
df= df.drop_duplicates()
print()
```

```
#to drop permntly
df= df.drop_duplicates(inplace = True)
print(df)
```

None

```
#keep the first occurrence of an duplicate value
df.drop_duplicates(keep = "first")
```

	EmpID	Name	City	Salary
0	101	Ravi	Hyderabad	30000
1	102	PRIYA	Hyderabad	40000
2	103	Arun	CHENNAI	3500
3	104	sneha	Chennai	4500
4	105	Kiran	Bangalore	32000
5	106	ANJALI	BANGALORE	50000
6	107	Vikram	Mumbai	38000
7	108	meena	mumbai	42000
8	109	RAHUL	Delhi	36000
9	110	sneha	Hyderabad	45000

```
#to keep the last occurence of an duplicate value
df.drop_duplicates(keep = "last")
```

	EmpID	Name	City	Salary
0	101	Ravi	Hyderabad	30000
1	102	PRIYA	Hyderabad	40000
2	103	Arun	CHENNAI	3500
3	104	sneha	Chennai	4500
4	105	Kiran	Bangalore	32000
5	106	ANJALI	BANGALORE	50000
6	107	Vikram	Mumbai	38000
7	108	meena	mumbai	42000
8	109	RAHUL	Delhi	36000
10	110	sneha	Hyderabad	45000

```
#to drop all the duplicate values
df.drop_duplicates(keep = False)
```

	EmpID	Name	City	Salary
0	101	Ravi	Hyderabad	30000
1	102	PRIYA	Hyderabad	40000
2	103	Arun	CHENNAI	3500
3	104	sneha	Chennai	4500
4	105	Kiran	Bangalore	32000
5	106	ANJALI	BANGALORE	50000
6	107	Vikram	Mumbai	38000
7	108	meena	mumbai	42000
8	109	RAHUL	Delhi	36000

```
#delete entire rows based on specific columns duplicates -- use subset
df = df.drop_duplicates(subset=["EmpID"],keep = "first") #subset -- check duplicates based on EmpID
print(df)
```

	EmpID	Name	City	Salary
0	101	Ravi	Hyderabad	30000
1	102	PRIYA	Hyderabad	40000
2	103	Arun	CHENNAI	3500
3	104	sneha	Chennai	4500
4	105	Kiran	Bangalore	32000
5	106	ANJALI	BANGALORE	50000
6	107	Vikram	Mumbai	38000
7	108	meena	mumbai	42000
8	109	RAHUL	Delhi	36000
9	110	sneha	Hyderabad	45000

```
#Dataset 1 - Customer Transactions
#This dataset represents customer purchase transactions.
```

```
import pandas as pd

data = {

    "TransactionID": [1001, 1002, 1003, 1004, 1005,

                      1006, 1007, 1008, 1009, 1010, 1010, 1011],

    "Customer": ["Ravi", "Priya", "Arun", "Sneha", "Kiran",

                 "Anjali", "Vikram", "Meena", "Rahul", "Sneha",

                 "Sneha", "Kiran"],

    "Product": ["Laptop", "Mobile", "Tablet", "Headphones", "Mouse",

                "Keyboard", "Monitor", "Laptop", "Mobile", "Headphones",

                "Headphones", "Mouse"],

    "Amount": [55000, 25000, 30000, 5000, 1500,

               2500, 18000, 55000, 22000, 5000,

               5000, 1500]}

df = pd.DataFrame(data)

print(df)
```

	TransactionID	Customer	Product	Amount
0	1001	Ravi	Laptop	55000
1	1002	Priya	Mobile	25000
2	1003	Arun	Tablet	30000
3	1004	Sneha	Headphones	5000
4	1005	Kiran	Mouse	1500
5	1006	Anjali	Keyboard	2500
6	1007	Vikram	Monitor	18000
7	1008	Meena	Laptop	55000
8	1009	Rahul	Mobile	22000
9	1010	Sneha	Headphones	5000
10	1010	Sneha	Headphones	5000
11	1011	Kiran	Mouse	1500

```
#display all the duplicate rows
print(df.duplicated())
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9    False
10   True
11   False
dtype: bool
```

```
#find the total no of dupl rows in the dataset
df = df.duplicated().sum()
print(df)
```

```
1
```

```
#display all records involved in the complete duplication
df = df[df.duplicated(keep = False)]
print(df)
```

	TransactionID	Customer	Product	Amount
9	1010	Sneha	Headphones	5000
10	1010	Sneha	Headphones	5000

```
#cal the total amt associated with the dupl tran records
duplicate_rows = df[df.duplicated(keep=False)]
total = duplicate_rows["Amount"].sum()
print(total)
```

```
10000
```

```
#check whether any transaction Id appears more than onces,display the duplicate transaction records
duplicate_rows = df[df.duplicated(keep=False)]
total = duplicate_rows["TransactionID"].sum()
print(total)
```

2020

```
#find all customers whose name appear more than onces in dataset.display there trans details
duplicate_rows = df[df["Customer"].duplicated(keep=False)]
print(duplicate_rows)
```

	TransactionID	Customer	Product	Amount
9	1010	Sneha	Headphones	5000
10	1010	Sneha	Headphones	5000

```
#remove cml duplicate transc from the dataset while keeping the first occurrence
df.drop_duplicates(subset="TransactionID", keep="first")
```

	TransactionID	Customer	Product	Amount
0	1001	Ravi	Laptop	55000
1	1002	Priya	Mobile	25000
2	1003	Arun	Tablet	30000
3	1004	Sneha	Headphones	5000
4	1005	Kiran	Mouse	1500
5	1006	Anjali	Keyboard	2500
6	1007	Vikram	Monitor	18000
7	1008	Meena	Laptop	55000
8	1009	Rahul	Mobile	22000
9	1010	Sneha	Headphones	5000
11	1011	Kiran	Mouse	1500

```
#remove records based on the dupl records
df=df.drop_duplicates(
    subset=["TransactionID"],keep="first")
print(df)
```

	TransactionID	Customer	Product	Amount
0	1001	Ravi	Laptop	55000
1	1002	Priya	Mobile	25000
2	1003	Arun	Tablet	30000
3	1004	Sneha	Headphones	5000
4	1005	Kiran	Mouse	1500
5	1006	Anjali	Keyboard	2500
6	1007	Vikram	Monitor	18000
7	1008	Meena	Laptop	55000
8	1009	Rahul	Mobile	22000
9	1010	Sneha	Headphones	5000
11	1011	Kiran	Mouse	1500

```
#9.remove customer and product keep the first occ
df = df.drop_duplicates(subset=["Customer", "Product"], keep="first")

print(df)
```

	TransactionID	Customer	Product	Amount
0	1001	Ravi	Laptop	55000
1	1002	Priya	Mobile	25000
2	1003	Arun	Tablet	30000
3	1004	Sneha	Headphones	5000
4	1005	Kiran	Mouse	1500
5	1006	Anjali	Keyboard	2500
6	1007	Vikram	Monitor	18000
7	1008	Meena	Laptop	55000
8	1009	Rahul	Mobile	22000

```
#10.crete a cleaned version of the sales dataframe where dupli tran id values are removed and only the first occ is retained

cleaned_df = df.drop_duplicates(subset="TransactionID", keep="first")
```

```
print(cleaned_df)
```

	TransactionID	Customer	Product	Amount
0	1001	Ravi	Laptop	55000
1	1002	Priya	Mobile	25000
2	1003	Arun	Tablet	30000
3	1004	Sneha	Headphones	5000
4	1005	Kiran	Mouse	1500
5	1006	Anjali	Keyboard	2500
6	1007	Vikram	Monitor	18000
7	1008	Meena	Laptop	55000
8	1009	Rahul	Mobile	22000
9	1010	Sneha	Headphones	5000
11	1011	Kiran	Mouse	1500

```
#Dataset 2 - Hospital Patient Visits
```

```
#This dataset represents patient visit information in a hospital.
```

```
import pandas as pd
```

```
data = {
```

```
    "VisitID": [501, 502, 503, 504, 505,
```

```
                506, 507, 508, 509, 510, 511],
```

```
    "Patient": ["Rahul", "Anita", "Suresh", "Priya", "Kiran",
```

```
                "Anjali", "Ravi", "Meena", "Arun", "Priya",
```

```
                "Priya", "Kiran"],
```

```
    "Department": ["Cardiology", "Neurology", "Orthopedic", "General",
```

```
                  "Dental", "Cardiology", "Neurology", "General",
```

```
                  "Dental", "Cardiology", "Cardiology", "Dental"],
```

```
    "BillAmount": [5000, 7500, 4500, 2000, 1500,
```

```
                  6000, 7000, 2500, 1800, 6000,6000, 1500]}}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

	VisitID	Patient	Department	BillAmount
0	501	Rahul	Cardiology	5000
1	502	Anita	Neurology	7500
2	503	Suresh	Orthopedic	4500
3	504	Priya	General	2000
4	505	Kiran	Dental	1500
5	506	Anjali	Cardiology	6000
6	507	Ravi	Neurology	7000
7	508	Meena	General	2500
8	509	Arun	Dental	1800
9	510	Priya	Cardiology	6000
10	510	Priya	Cardiology	6000
11	511	Kiran	Dental	1500

```
#1.
```

```
print(df.duplicated())
```

```
0    False
1    False
2    False
3    False
4    False
5    False
6    False
7    False
8    False
9    False
10   True
11   False
dtype: bool
```

```
#2.
```

```
df = df.duplicated().sum()
```

```
print(df)
```

```
1
```

```
#3.
print(df[df.duplicated(keep = False)])
```

	VisitID	Patient	Department	BillAmount
9	510	Priya	Cardiology	6000
10	510	Priya	Cardiology	6000

```
#4.
duplicate_rows = df[df.duplicated(keep=False)]
total = duplicate_rows["BillAmount"].sum()
print(total)
```

12000

```
#5.
duplicate_rows = df[df.duplicated(keep=False)]
total = duplicate_rows["VisitID"].sum()
print(total)
```

1020

```
#7.
df.drop_duplicates(subset="VisitID", keep="first")
```

	VisitID	Patient	Department	BillAmount
0	501	Rahul	Cardiology	5000
1	502	Anita	Neurology	7500
2	503	Suresh	Orthopedic	4500
3	504	Priya	General	2000
4	505	Kiran	Dental	1500
5	506	Anjali	Cardiology	6000
6	507	Ravi	Neurology	7000
7	508	Meena	General	2500
8	509	Arun	Dental	1800
9	510	Priya	Cardiology	6000
11	511	Kiran	Dental	1500

```
#Basic groupby() -- suppose we have emp data with diff cities
df.groupby("Department")["BillAmount"].mean()
```

BillAmount	
Department	
Cardiology	5750.0
Dental	1600.0
General	2250.0
Neurology	7250.0
Orthopedic	4500.0

dtype: float64

```
df.groupby("Department")["BillAmount"].max()
```

BillAmount	
Department	
Cardiology	6000
Dental	1800
General	2500
Neurology	7500
Orthopedic	4500

dtype: int64

```
df.groupby("Department")["BillAmount"].min()
```

BillAmount	
Department	
Cardiology	5000
Dental	1500
General	2000
Neurology	7000
Orthopedic	4500

dtype: int64

```
df.groupby("Department")["BillAmount"].count()
```

BillAmount	
Department	
Cardiology	4
Dental	3
General	2
Neurology	2
Orthopedic	1

dtype: int64

```
#multiple calc --- helps to perform multiple agg calc
df.groupby("Department")["BillAmount"].agg([
    'count', 'mean', 'max', 'min'])
```

	count	mean	max	min
Department				
Cardiology	4	5750.0	6000	5000
Dental	3	1600.0	1800	1500
General	2	2250.0	2500	2000
Neurology	2	7250.0	7500	7000
Orthopedic	1	4500.0	4500	4500

```
#multiple aggregations
import pandas as pd
data = {
    "EmpID" : [101,102,103,104,105],
    "Name" : ["Ravi", "PRIYA", "Arun", "sneha", "Kiran"],
    "City" : ["Hyderabad", "Hyderabad", "CHENNAI", "Chennai", "Bangalore"],
    "Salary" : [30000, 40000, 3500, 4500, 32000],
    "Age" : [22, 23, 34, 45, 24]
}

df = pd.DataFrame(data)
print(df)

df.groupby("City").agg({
    "Age" : ['mean', 'max'],
    "Salary" : ['sum', 'count']
})
```

	EmpID	Name	City	Salary	Age
0	101	Ravi	Hyderabad	30000	22
1	102	PRIYA	Hyderabad	40000	23
2	103	Arun	CHENNAI	3500	34
3	104	sneha	Chennai	4500	45
4	105	Kiran	Bangalore	32000	24

	Age	Salary		
	mean	max	sum	count
City				
Bangalore	24.0	24	32000	1
CHENNAI	34.0	34	3500	1
Chennai	45.0	45	4500	1
Hyderabad	22.5	23	70000	2

```
#missing inconsistent data
import pandas as pd
data = {
    "EmpID" : [101,102,103,104,105,106,107,108,109,110,110],
    "Name" : ["Ravi", "PRIYA", "Arun", "sneha", "Kiran", "ANJALI", "Vikram", "meena", "RAHUL", "Sneha", "Sneha"],
    "City" : ["Hyderabad", "Hyderabad", "CHENNAI", "Chennai", "Bangalore", "BANGALORE", "Mumbai", "mumbai", "Delhi", "Hyderabad", "Hyderabad"],
    "Salary" : [30000,40000,3500,4500,32000,50000,38000,42000,36000,45000,45000],
}
df = pd.DataFrame(data)

df["Name"] = df["Name"].str.title()
print(df)

print("-----")

df["City"] = df["City"].str.title()
print(df)
```

	EmpID	Name	City	Salary
0	101	Ravi	Hyderabad	30000
1	102	Priya	Hyderabad	40000
2	103	Arun	CHENNAI	3500
3	104	Sneha	Chennai	4500
4	105	Kiran	Bangalore	32000
5	106	Anjali	BANGALORE	50000
6	107	Vikram	Mumbai	38000
7	108	Meena	mumbai	42000
8	109	Rahul	Delhi	36000
9	110	Sneha	Hyderabad	45000
10	110	Sneha	Hyderabad	45000

	EmpID	Name	City	Salary
0	101	Ravi	Hyderabad	30000
1	102	Priya	Hyderabad	40000
2	103	Arun	Chennai	3500
3	104	Sneha	Chennai	4500
4	105	Kiran	Bangalore	32000
5	106	Anjali	Bangalore	50000
6	107	Vikram	Mumbai	38000
7	108	Meena	Mumbai	42000
8	109	Rahul	Delhi	36000
9	110	Sneha	Hyderabad	45000
10	110	Sneha	Hyderabad	45000

```
df["Name"] = df["Name"].str.upper()
print(df)

print("-----")

df["City"] = df["City"].str.upper()
print(df)
```

	EmpID	Name	City	Salary
0	101	RAVI	Hyderabad	30000
1	102	PRIYA	Hyderabad	40000
2	103	ARUN	Chennai	3500
3	104	SNEHA	Chennai	4500
4	105	KIRAN	Bangalore	32000
5	106	ANJALI	Bangalore	50000
6	107	VIKRAM	Mumbai	38000
7	108	MEENA	Mumbai	42000
8	109	RAHUL	Delhi	36000
9	110	SNEHA	Hyderabad	45000
10	110	SNEHA	Hyderabad	45000

```

-----
      EmpID   Name      City  Salary
0      101    RAVI  HYDERABAD  30000
1      102   PRIYA  HYDERABAD  40000
2      103    ARUN   CHENNAI   3500
3      104   SNEHA   CHENNAI   4500
4      105   KIRAN  BANGALORE  32000
5      106  ANJALI  BANGALORE  50000
6      107  VIKRAM    MUMBAI  38000
7      108   MEENA    MUMBAI  42000
8      109   RAHUL    DELHI   36000
9      110   SNEHA  HYDERABAD  45000
10     110   SNEHA  HYDERABAD  45000

```

```

df["Name"] = df["Name"].str.lower()
print(df)

print("-----")

df["City"] = df["City"].str.lower()
print(df)

```

```

      EmpID   Name      City  Salary
0      101    ravi  Hyderabad  30000
1      102   priya  Hyderabad  40000
2      103    arun   Chennai   3500
3      104   sneha   Chennai   4500
4      105   kiran  Bangalore  32000
5      106  anjali  Bangalore  50000
6      107  vikram    Mumbai  38000
7      108   meena    Mumbai  42000
8      109   rahul    Delhi   36000
9      110   sneha  Hyderabad  45000
10     110   sneha  Hyderabad  45000
-----
      EmpID   Name      City  Salary
0      101    ravi  hyderabad  30000
1      102   priya  hyderabad  40000
2      103    arun   chennai   3500
3      104   sneha   chennai   4500
4      105   kiran  bangalore  32000
5      106  anjali  bangalore  50000
6      107  vikram    mumbai  38000
7      108   meena    mumbai  42000
8      109   rahul    delhi   36000
9      110   sneha  hyderabad  45000
10     110   sneha  hyderabad  45000

```

```

df["Name"] = df["Name"].str.capitalize()
print(df)

print("-----")

df["City"] = df["City"].str.capitalize()
print(df)

```

```

      EmpID   Name      City  Salary
0      101    Ravi  Hyderabad  30000
1      102   Priya  Hyderabad  40000
2      103    Arun   Chennai   3500
3      104   Sneha   Chennai   4500
4      105   Kiran  Bangalore  32000
5      106  Anjali  Bangalore  50000
6      107  Vikram    Mumbai  38000
7      108   Meena    Mumbai  42000
8      109   Rahul    Delhi   36000
9      110   Sneha  Hyderabad  45000
10     110   Sneha  Hyderabad  45000
-----
      EmpID   Name      City  Salary
0      101    Ravi  Hyderabad  30000
1      102   Priya  Hyderabad  40000
2      103    Arun   Chennai   3500
3      104   Sneha   Chennai   4500
4      105   Kiran  Bangalore  32000
5      106  Anjali  Bangalore  50000
6      107  Vikram    Mumbai  38000
7      108   Meena    Mumbai  42000
8      109   Rahul    Delhi   36000
9      110   Sneha  Hyderabad  45000
10     110   Sneha  Hyderabad  45000

```

```

#saturday - 22/08/26
import pandas as pd
data = {
    "OrderID": [

```

```
1001, 1002, 1003, 1004, 1005,
1006, 1007, 1008, 1009, 1010,
1011, 1012, 1013, 1014, 1015
],

"OrderDateTime": [
    "2024-01-05 09:15:20",
    "2024-02-10 14:30:45",
    "2024-03-15 18:20:10",
    "2024-04-20 11:45:30",
    "2024-05-25 20:10:55",
    "2024-06-12 08:35:15",
    "2024-07-18 16:25:40",
    "2024-08-22 19:50:25",
    "2024-09-05 10:20:35",
    "2024-10-14 13:40:50",
    "2024-11-20 17:15:10",
    "2024-12-25 21:30:45",
    "2025-01-10 09:45:20",
    "2025-02-15 15:10:35",
    "2025-03-20 18:55:50"
],

"Product": [
    "Laptop", "Mobile", "Tablet", "Headphones", "Keyboard",
    "Mouse", "Monitor", "Laptop", "Mobile", "Tablet",
    "Headphones", "Laptop", "Keyboard", "Monitor", "Mobile"
],

"Sales": [
    55000, 25000, 30000, 5000, 3500,
    1500, 18000, 60000, 27000, 32000,
    6500, 58000, 4000, 20000, 29000
]
}

df = pd.DataFrame(data)

print(df)
```

	OrderID	OrderDateTime	Product	Sales
0	1001	2024-01-05 09:15:20	Laptop	55000
1	1002	2024-02-10 14:30:45	Mobile	25000
2	1003	2024-03-15 18:20:10	Tablet	30000
3	1004	2024-04-20 11:45:30	Headphones	5000
4	1005	2024-05-25 20:10:55	Keyboard	3500
5	1006	2024-06-12 08:35:15	Mouse	1500
6	1007	2024-07-18 16:25:40	Monitor	18000
7	1008	2024-08-22 19:50:25	Laptop	60000
8	1009	2024-09-05 10:20:35	Mobile	27000
9	1010	2024-10-14 13:40:50	Tablet	32000
10	1011	2024-11-20 17:15:10	Headphones	6500
11	1012	2024-12-25 21:30:45	Laptop	58000
12	1013	2025-01-10 09:45:20	Keyboard	4000
13	1014	2025-02-15 15:10:35	Monitor	20000
14	1015	2025-03-20 18:55:50	Mobile	29000

```
#friday-- H.w
#Dataset 2 - Hospital Patient Visits
#This dataset represents patient visit information in a hospital.

data = {

    "VisitID": [501, 502, 503, 504, 505,
                506, 507, 508, 509, 510, 511],

    "Patient": ["Rahul", "Anita", "Suresh", "Priya", "Kiran",
                "Anjali", "Ravi", "Meena", "Arun", "Priya",
                "Priya", "Kiran"],

    "Department": ["Cardiology", "Neurology", "Orthopedic", "General",
                   "Dental", "Cardiology", "Neurology", "General",
                   "Dental", "Cardiology", "Cardiology", "Dental"],

    "BillAmount": [5000, 7500, 4500, 2000, 1500,
                   6000, 7000, 2500, 1800, 6000, 6000, 1500]}
```

```
df_patient = pd.DataFrame(data)
```

```
print(df_patient)
```

	VisitID	Patient	Department	BillAmount
0	501	Rahul	Cardiology	5000
1	502	Anita	Neurology	7500
2	503	Suresh	Orthopedic	4500
3	504	Priya	General	2000
4	505	Kiran	Dental	1500
5	506	Anjali	Cardiology	6000
6	507	Ravi	Neurology	7000
7	508	Meena	General	2500
8	509	Arun	Dental	1800
9	510	Priya	Cardiology	6000
10	510	Priya	Cardiology	6000
11	511	Kiran	Dental	1500

```
# 1. Remove spaces from Patient
```

```
df_patient["Patient"] = df_patient["Patient"].str.strip()
```

```
print("-----")
```

```
# 2. Convert Patient to lowercase
```

```
df_patient["Patient"] = df_patient["Patient"].str.lower()
```

```
print("-----")
```

```
# 3. Convert Patient to title case
```

```
df_patient["Patient"] = df_patient["Patient"].str.title()
```

```
print("-----")
```

```
# 4. Department to lowercase
```

```
df_patient["Department"] = df_patient["Department"].str.strip().str.lower()
```

```
print("-----")
```

```
# 5. Department to title case
```

```
df_patient["Department"] = df_patient["Department"].str.title()
```

```
print("-----")
```

```
# 6. Find Cardiology patients
```

```
cardiology = df_patient[
    df_patient["Department"].str.strip().str.lower() == "cardiology"
]
print(cardiology)
```

```
print("-----")
```

```
# 7. Unique departments
```

```
print("Unique departments:")
print(df_patient["Department"].unique())
```

```
print("-----")
```

```
# 8. Display cleaned DataFrame
```

```
print("Cleaned DataFrame:")
print(df_patient)
```

```
print("-----")
```

```
# 9. Find repeated patients
```

```
duplicate_patients = df_patient[
    df_patient["Patient"].duplicated(keep=False)
]
print("Repeated patients:")
print(duplicate_patients)
```

```
print("-----")
```

```
Department
Cardiology      4
Dental          3
Neurology       2
General         2
Orthopedic      1
Name: count, dtype: int64
```

```
print("-----")
```

[illegible]

```
import pandas as pd
import numpy as np
```

```
#Q1. Generate 5 years of monthly dates from 2020-01-01 , 5 years × 12 months = 60 months.
dates = pd.date_range(start="2020-01-01", periods=60, freq="MS")

df = pd.DataFrame({
    "Date": dates,
    "Sales": np.random.randint(1000, 10000, size=60)
})

print(df)
```

	Date	Sales
0	2020-01-01	9557
1	2020-02-01	5046
2	2020-03-01	7859
3	2020-04-01	4186
4	2020-05-01	7177
5	2020-06-01	5385
6	2020-07-01	4337
7	2020-08-01	6367
8	2020-09-01	5503
9	2020-10-01	3383
10	2020-11-01	1960
11	2020-12-01	1820
12	2021-01-01	1532
13	2021-02-01	9553
14	2021-03-01	5000
15	2021-04-01	4793
16	2021-05-01	8475
17	2021-06-01	2003
18	2021-07-01	8069
19	2021-08-01	5318
20	2021-09-01	3523
21	2021-10-01	3436
22	2021-11-01	8430
23	2021-12-01	1434
24	2022-01-01	3698
25	2022-02-01	3544
26	2022-03-01	7261
27	2022-04-01	5547
28	2022-05-01	9285
29	2022-06-01	1735
30	2022-07-01	4385
31	2022-08-01	9459
32	2022-09-01	9004
33	2022-10-01	6616
34	2022-11-01	7256
35	2022-12-01	6575
36	2023-01-01	5849
37	2023-02-01	6773
38	2023-03-01	4364
39	2023-04-01	1144
40	2023-05-01	8716
41	2023-06-01	7525
42	2023-07-01	4653
43	2023-08-01	2704
44	2023-09-01	1035
45	2023-10-01	4406
46	2023-11-01	4005
47	2023-12-01	4559
48	2024-01-01	7645
49	2024-02-01	8696
50	2024-03-01	4767
51	2024-04-01	9685
52	2024-05-01	3548
53	2024-06-01	3765
54	2024-07-01	3346
55	2024-08-01	7559
56	2024-09-01	3833

```
#Q2. Generate 365 daily dates from 2024-01-01
#Here 10001 is used because the upper limit of np.random.randint() is excluded, so this generates values from ₹1,000 to ₹10,000
dates = pd.date_range(start="2024-01-01", periods=365, freq="D")

df = pd.DataFrame({
    "Date": dates,
    "Sales": np.random.randint(1000, 10001, size=365)
})

print(df)
```

	Date	Sales
0	2024-01-01	8345
1	2024-01-02	6367

```
2    2024-01-03    8758
3    2024-01-04    9936
4    2024-01-05    4855
..
360 2024-12-26    3015
361 2024-12-27    7115
362 2024-12-28    6958
363 2024-12-29    4110
364 2024-12-30    1184
```

[365 rows x 2 columns]

```
#Q3. Generate 24 months of monthly sales from 2023-01-01
#This gives:
```

```
#2023 → 12 months
#2024 → 12 months
#Total → 24 months
```

```
dates = pd.date_range(start="2023-01-01", periods=24, freq="MS")
```

```
df = pd.DataFrame({
    "Date": dates,
    "Sales": np.random.randint(1000, 10001, size=24)
})
```

```
print(df)
```

	Date	Sales
0	2023-01-01	6146
1	2023-02-01	5538
2	2023-03-01	6006
3	2023-04-01	4670
4	2023-05-01	8962
5	2023-06-01	3549
6	2023-07-01	8050
7	2023-08-01	7695
8	2023-09-01	1809
9	2023-10-01	4682
10	2023-11-01	7941
11	2023-12-01	8053
12	2024-01-01	5462
13	2024-02-01	5563
14	2024-03-01	6169
15	2024-04-01	2825
16	2024-05-01	8890
17	2024-06-01	5758
18	2024-07-01	5899
19	2024-08-01	6922
20	2024-09-01	1711
21	2024-10-01	5511
22	2024-11-01	7325
23	2024-12-01	7093

```
#Q4. Generate 30 days of hourly sales      30 days × 24 hours = 720 hours.  --freq="h" → hourly frequency.
```

```
dates = pd.date_range(
    start="2024-01-01 00:00:00",
    periods=720,
    freq="h"
)
```

```
df = pd.DataFrame({
    "Date": dates,
    "Sales": np.random.randint(1000, 10001, size=720)
})
```

```
print(df)
```

	Date	Sales
0	2024-01-01 00:00:00	1633
1	2024-01-01 01:00:00	7434
2	2024-01-01 02:00:00	9115
3	2024-01-01 03:00:00	6405
4	2024-01-01 04:00:00	5630
..	..	..
715	2024-01-30 19:00:00	2022
716	2024-01-30 20:00:00	6425
717	2024-01-30 21:00:00	1225
718	2024-01-30 22:00:00	1812
719	2024-01-30 23:00:00	1597

[720 rows x 2 columns]

```
#Q5. Generate 5 years of monthly sales and find yearly total
dates = pd.date_range(start="2020-01-01", periods=60, freq="MS")
```

```

df = pd.DataFrame({
    "Date": dates,
    "Sales": np.random.randint(1000, 10001, size=60)
})

# Extract Year and Month
df["Year"] = df["Date"].dt.year
df["Month"] = df["Date"].dt.month

# Total sales for each year
yearly_sales = df.groupby("Year")["Sales"].sum()

print(df)
print(yearly_sales)

```

	Date	Sales	Year	Month
0	2020-01-01	5212	2020	1
1	2020-02-01	4692	2020	2
2	2020-03-01	6628	2020	3
3	2020-04-01	1726	2020	4
4	2020-05-01	7032	2020	5
5	2020-06-01	7485	2020	6
6	2020-07-01	7231	2020	7
7	2020-08-01	2711	2020	8
8	2020-09-01	6491	2020	9
9	2020-10-01	3615	2020	10
10	2020-11-01	5662	2020	11
11	2020-12-01	7782	2020	12
12	2021-01-01	6419	2021	1
13	2021-02-01	6046	2021	2
14	2021-03-01	7994	2021	3
15	2021-04-01	7583	2021	4
16	2021-05-01	2408	2021	5
17	2021-06-01	6231	2021	6
18	2021-07-01	3985	2021	7
19	2021-08-01	9446	2021	8
20	2021-09-01	4341	2021	9
21	2021-10-01	4245	2021	10
22	2021-11-01	9608	2021	11
23	2021-12-01	5402	2021	12
24	2022-01-01	4304	2022	1
25	2022-02-01	4768	2022	2
26	2022-03-01	3181	2022	3
27	2022-04-01	1465	2022	4
28	2022-05-01	3918	2022	5
29	2022-06-01	4939	2022	6
30	2022-07-01	6690	2022	7
31	2022-08-01	2651	2022	8
32	2022-09-01	4941	2022	9
33	2022-10-01	8670	2022	10
34	2022-11-01	1825	2022	11
35	2022-12-01	7556	2022	12
36	2023-01-01	4377	2023	1
37	2023-02-01	8060	2023	2
38	2023-03-01	3315	2023	3